Entornos de Desarrollo

FICHA TÉCNICA DE LA UNIDAD

Unidad trabajo	UT5. Entornos de desarrollo
Resultados de Aprendizaje	RA5. Utiliza entornos de desarrollo integrados y sistemas de control de versiones para depurar, probar e integrar código de forma colaborativa.
Criterios de Evaluación	CE5.a: Utilizar herramientas integradas de depuración (debuggers) para inspeccionar variables. CE5.b: Automatizar pruebas unitarias. CE5.c: Resolver proyectos colaborativos empleando Git y repositorios remotos. CE5.d: Aplicar metodologías ágiles.
Tiempo	6 horas

ÍNDICE DE CONTENIDOS

1. Introducción

2. ¿Qué es Visual Studio Code?

3. El IDE: Configuración Básica de VS Code

4. Gestión de Entornos Virtuales

5. Depuración Visual (Debugging) sin "Prints"

6. Control de Versiones con Git

7. Colaboración Remota con GitHub

Introducción

Hoy en día, un administrador de sistemas moderno no trabaja a ciegas. Aunque es vital conocer la terminal, herramientas avanzadas como **Visual Studio Code (VS Code)** nos permiten desarrollar, depurar y versionar nuestros scripts de automatización de forma visual, reduciendo errores humanos y aumentando la velocidad de despliegue.

Para esta unidad, nos centraremos en realizar todas las operaciones utilizando la **interfaz gráfica (GUI)** de VS Code. Esto nos facilitará entender la lógica del control de versiones y el aislamiento de código sin pelearnos con comandos complejos en la terminal de manera inicial.

2. ¿Qué es Visual Studio Code?

Visual Studio Code (VS Code) es un editor de código fuente ligero y potente desarrollado por Microsoft, diseñado para ser rápido, multiplataforma y altamente personalizable.

Características importantes

- **IntelliSense**: Ofrece autocompletado inteligente de código basado en tipos de variables, definiciones de funciones y módulos importados. Proporciona sugerencias de sintaxis y documentación rápida al pasar el cursor sobre los elementos.
- **Depuración integrada**: Incluye herramientas nativas que permiten establecer puntos de interrupción, inspeccionar variables y analizar la pila de llamadas sin abandonar el editor, lo que optimiza la corrección de errores.
- **Integración con Git**: Permite la gestión visual de repositorios, facilitando operaciones como commits, staging y sincronización (push/pull) directamente desde la interfaz.
- **Ecosistema de extensiones**: Acceso a miles de complementos para añadir soporte a lenguajes, temas personalizados y herramientas adicionales que transforman el editor en un entorno de desarrollo completo.

Descripción de la interfaz

- **Barra de actividad (izquierda)**: Punto de acceso principal para navegar entre el explorador de archivos, la búsqueda global, el control de código fuente y el gestor de extensiones.
- **Explorador (sidebar)**: Proporciona una vista jerárquica de los archivos y carpetas del proyecto actual, permitiendo una organización eficiente.
- **Área de edición**: El espacio central donde se desarrolla la actividad principal de escritura y edición de código.
- **Panel inferior**: Contiene pestañas esenciales como la Terminal integrada, la consola de depuración y el panel de salida de errores.

- **Barra de estado (inferior)**: Muestra información relevante sobre el entorno activo, el lenguaje de programación detectado, la codificación y el estado del repositorio Git.

3. El IDE: Configuración Básica de VS Code

Asistentes Visuales de Programación

Al abrir un script de Python en VS Code, el editor utiliza sus extensiones instaladas para ofrecernos ayuda visual instantánea:

- **Pylance (Autocompletado)**: Al escribir código, nos sugiere comandos, nombres de variables y funciones. Si pasamos el cursor sobre una función, nos muestra una ventana flotante explicando qué hace y qué parámetros necesita.
- **Análisis Dinámico (Errores en vivo)**: Si cometemos un error de sintaxis (como olvidar un paréntesis), la pestaña **Problemas** (*Problems*) en la parte inferior nos mostrará el error exacto y la línea afectada con un subrayado ondulado rojo.

Formateo Automático con un Clic

Para que nuestro código cumpla las reglas profesionales de estilo (PEP 8) sin que tengamos que contar espacios manualmente, configuramos el formateador visual de VS Code:

1. Abrimos las **Settings (Configuración)** usando el icono de engranaje (abajo a la izquierda) o con Ctrl + ,.
2. Buscamos Format On Save y marcamos la casilla.
3. Ahora, cada vez que guardemos el archivo (Ctrl + S), VS Code reordenará la indentación, añadirá los espacios correctos y dejará el script perfectamente formateado de manera automática.

4. Gestión de Entornos Virtuales

Como administradores, debemos garantizar que las librerías adicionales que instalemos para un script (por ejemplo, para conectarnos a una base de datos) no interfieran con las herramientas del sistema operativo. Un **entorno virtual** es simplemente una carpeta local (normalmente llamada .venv) donde se guardan estas librerías de forma aislada.

Creación y Selección Visual del Entorno

Paso 1: Crear la carpeta local .venv

Aunque la creación requiere un comando inicial sencillo en la terminal del proyecto:

python -m venv .venv

Una vez creado, **no necesitaremos usar más la consola** para su configuración.

Paso 2: El aviso inteligente de VS Code

Nada más crear la carpeta, VS Code detectará el entorno virtual y nos mostrará un aviso flotante abajo a la derecha:

"Hemos detectado que se ha creado un nuevo entorno virtual. ¿Desea seleccionarlo para la carpeta de trabajo?"

Hacemos clic en **Sí (Yes)**.

Paso 3: Selección manual del intérprete (Si no sale el aviso)

Si no aparece el aviso o queremos cambiarlo manualmente:

1. Hacemos clic en la versión de Python que aparece en la **barra de estado inferior derecha** (ej. *3.10.x* o *Select Interpreter*).
2. O bien, abrimos la paleta de comandos con Ctrl + Shift + P, escribimos Python: Select Interpreter y pulsamos Enter.
3. En la lista desplegable, seleccionamos la opción que lleva la etiqueta **('.venv': venv)**.

Paso 4: Activación automática en la terminal integrada

Al abrir una nueva terminal en VS Code (Ctrl + Shift + \``), el editor detectará nuestra configuración visual y activará el entorno automáticamente, mostrando el prefijo (.venv)` al inicio de la línea de comandos sin tener que escribir instrucciones de activación.

5. Depuración Visual (Debugging) sin "Prints"

La depuración visual nos permite pausar el script en pleno funcionamiento y examinar la memoria del ordenador sin necesidad de llenar el código de sentencias print() que luego tendríamos que borrar.

Configuración del Depurador

1. Hacemos clic en el icono de **Run and Debug (Ejecutar y depurar)** en la barra lateral izquierda (el icono de un escarabajo con un botón de "Play").
2. Hacemos clic en el botón de **Run and Debug** y seleccionamos **Python File**.

El Punto de Ruptura (Breakpoint)

- **¿Qué es?**: Una señal que le indica al ordenador: *"Detente aquí antes de ejecutar esta línea para que pueda examinar qué está pasando"*.
- **Cómo colocarlo**: Movemos el cursor justo a la izquierda del número de la línea de código donde queremos pausar. Veremos un **punto rojo translúcido**. Hacemos clic y el punto se volverá de color rojo brillante y fijo.

Panel de Control de Depuración (Barra Flotante)

Al iniciarse la depuración y alcanzar el punto de ruptura, el script se pausará y aparecerá una barra de herramientas flotante en la parte superior:

1. **Continue (F5)**: Reanuda la ejecución del programa de forma normal hasta el siguiente punto de ruptura o hasta el final del script.
2. **Step Over (F10)**: Pasa a la siguiente línea de código de este archivo. Ideal para ir avanzando línea a línea observando los cambios.
3. **Step Into (F11)**: Si la línea actual llama a una función que hemos programado nosotros, el depurador "entra" dentro de ella para ver qué pasa en su interior.
4. **Step Out (Shift + F11)**: Si estamos dentro de una función, ejecuta el resto de la misma de golpe y vuelve al flujo principal.
5. **Restart (Ctrl + Shift + F5)**: Detiene el script y lo vuelve a arrancar desde el principio con la memoria limpia.
6. **Stop (Shift + F5)**: Detiene por completo la ejecución y el análisis.

Inspección Visual de Datos

Mientras el script está pausado (en amarillo), podemos utilizar los paneles de la izquierda de la pantalla:

- **Panel de Variables**: Muestra de forma estructurada todas las variables creadas hasta el momento y su valor actual. Podemos hacer clic sobre ellas y cambiar su valor en directo para probar diferentes casos de prueba.
- **Panel Watch (Inspección)**: Hacemos clic en el botón + y escribimos una condición lógica (ej. precio > 100). El sistema nos dirá en directo si esa condición es Verdadera o Falsa a medida que avanzamos línea a línea.

6. Control de Versiones con Git

VS Code cuenta con una pestaña dedicada en la barra lateral izquierda llamada **Control de Código Fuente (Source Control)**. Desde aquí realizaremos todo el ciclo de Git de forma 100% visual.

Ciclo de Trabajo en Git mediante la Interfaz

1. Inicializar el Repositorio

En lugar de escribir git init en consola:

1. Entramos en la pestaña de **Control de Código Fuente** (icono con tres nodos conectados).
2. Hacemos clic en el gran botón azul que dice **Initialize Repository (Inicializar repositorio)**.

2. El Staging Area (Preparar cambios)

- En nuestro panel de control de código fuente veremos el listado de archivos modificados bajo el título **Changes (Cambios)**.
- Para preparar un archivo (el equivalente a git add), simplemente pasamos el cursor sobre el archivo en la lista y hacemos clic en el icono **+** (*Stage Changes*) que aparece a su derecha. El archivo subirá a una sección llamada **Staged Changes (Cambios preparados)**.

3. El Commit (Registrar versión)

- En la caja de texto superior que dice *Message (Mensaje)*, escribimos de forma descriptiva qué cambio hace nuestro script.
- Hacemos clic en el botón azul superior **Commit** (o en el icono de la marca de verificación ✓). ¡Listo! Se ha creado el punto de control local.

El Archivo .gitignore

Para evitar que Git rastree archivos innecesarios (como la carpeta .venv o temporales de VS Code), hacemos lo siguiente de forma visual:

1. Creamos un nuevo archivo llamado exactamente .gitignore en el explorador de archivos.
2. Añadimos las carpetas que queremos descartar:

```
.venv/
__pycache__/
*.pyc
```

3. Guardamos. Veremos que la carpeta .venv cambia de color en nuestro explorador (se vuelve grisácea), indicando que Git ya la ignora.

7. Colaboración Remota con GitHub

Una de las mayores ventajas de VS Code es su integración nativa con GitHub, que elimina por completo el uso de terminales y la gestión manual de claves criptográficas SSH para principiantes.

Publicar en GitHub con un Clic

Una vez que hemos hecho nuestro primer Commit de forma local:

1. En la pestaña de **Control de Código Fuente**, aparecerá un botón azul llamado **Publish Branch (Publicar rama)**.
2. Al pulsarlo, VS Code nos preguntará si deseamos iniciar sesión en GitHub. Hacemos clic en **Permitir (Allow)**.
3. Se abrirá el navegador web para iniciar sesión de forma segura y autorizar la conexión.
4. Tras autorizar, volvemos a VS Code y elegimos una opción en la parte superior:

    - *Publish to GitHub private repository* (Repositorio privado).
    - *Publish to GitHub public repository* (Repositorio público).
5. VS Code creará el repositorio automáticamente en nuestra cuenta de GitHub y subirá todo nuestro código.

Sincronización Diaria (Push / Pull)

A partir de ese momento, el botón azul cambiará su nombre a **Sync Changes (Sincronizar cambios)**.

- Al pulsarlo, VS Code realizará de manera síncrona la descarga de cambios del servidor remoto (*Pull*) y la subida de nuestros nuevos commits locales (*Push*) con total comodidad y seguridad.